Practical Work 2: "Design and Implementation of a Data Warehouse for a 'Domain'"

This document outlines the second practical work assignment for the course "Data models in database systems". The assignment has been created to emphasize design, complex query creation, and comprehensive reporting, making it a group-based project.

Working with Your Domain Description

You will be given a domain description of 300-350 words (e.g., "Retail Analytics," "Flight Operations," "Online Learning Platform"). It will describe the main business processes, entities, and the types of analytical questions the business wants to answer.

Please note that the domain description is **intentionally vague**. It is not a precise requirements specification. Your group's task is to interpret this description, make informed design decisions, and add necessary details to create a functional data warehouse. You should:

- Add attributes and details where they are missing – but be careful not to contradict the domain.
- Define hierarchies within the data (e.g., geographical regions, time periods).
- Create formulas for calculated measures (e.g., profit, growth rate).

You will be graded on the justification for your design choices, the complexity of the structures you implement, and the semantic sense of your analytical queries.

Submissions

The submission deadline is **November 30**. An emergency extension may be requested via email before the deadline if you have not used it before for PW1. This is a **group project** for **2-3** students. While you are allowed to do it alone, it is not recommended.

You must submit three items to ORTUS:

1. **Report (*.pdf format)**: This document will contain your design, explanations, and a link to your video.
2. *Commented .sql file*: Your complete, commented code used to create and query the warehouse.
3. **Group Presentation Video**: A 5-7 minute video where all members explain the design and demonstrate the functionality of the warehouse, with each student presenting their specific contributions.

Grading and Points Distribution

The total project is worth **100 points**, distributed as follows:

- **Main Report & SQL File (80 points)**: This grade is based on the quality, correctness, and depth of the 5 tasks, broken down as:
    - **TASK 1 (DESIGN):** 20 points
        - Star Schema (10 points)
        - Design Justification (10 points)
            - Justification must include the goal of the data warehouse
    - **TASK 2 (ETL):** 10 points
        - Tables are created and populated correctly (5 points)
        - A PL/SQL procedure is created to automate the loading process (5 points)
    - **TASK 3 (QUERIES):** 20 points
        - ROLLUP / CUBE queries (8 points, 2 each): Do they work? Are these queries non-trivial?
        - PL/SQL Functions (8 points, 4 each)
        - MODEL usage (4 points)
    - **TASK 4 (AGGREGATE TABLE):** 20 points
        - Design & DDL (5 points)
        - Sample query (5 points)
        - Population Procedure (5 points)
        - Explain plan (5 points)
    - **REPORT STRUCTURE & CONCLUSIONS:** 10 points
        - The report structure is followed. Everything necessary in the screenshots is fully visible and legible. (3 points)
        - Conclusions acquired from doing the practical tasks as well as a reflection on how the group worked together. (7 points)
            - Points awarded will be reduced for surface level conclusions. For example, "the task was done successfully" and "task 1, task 2 was done successfully and everything worked out" will not be counted as a meaningful conclusion.
- **Group Presentation Video (20 points)**: This grade is based on the clarity of your explanations, the professionalism of the video, and the participation of all group members. Graded for each student individually.

- Clarity and professionalism (5 points)
- Design explanation (5 points)
- Live demonstration (5 points)
- Participation (5 points)

- **CRITICAL FAILURES OF THE TASK**
  - **0 for the entire project**
    - Plagiarism of any kind
    - Project not submitted by the final deadline
  - ***.sql file is not submitted**
    - ALL TASKS related to implementation (2, 3 and 4) receive a 0
  - If the created schema is not a warehouse schema:
    - Task 1 is an automatic 0
    - Because the rest of the tasks require a warehouse schema it is very likely that they will also be incorrect and also receive 0.
  - If the quality of a group members work is vastly different from other group members, then the final grade is subject to a large grade deduction or boost.
  - If someone in your group is not doing any part of the groupwork, send a message about it as soon as possible and adjustments will be made.
- **General task failures:**
  - Not following task requirements is graded with 0 points for each subtask.
  - Not following report structure is graded with 0 for each missing report and task.
  - Trivial queries and non-functional code will result in a score of 0 for each wrong task.
  - No justification given for written queries results in a score of 0 for the query.
  - No explanations for ANY of your written script result in a 0 for the corresponding script.
  - It is possible to lose points for other significant errors in the code or report not mentioned in this document. These will be mentioned when you receive your grade.

Task Software

To do this task you will need to download **Oracle Virtualbox** and follow the provided tutorial to set up a virtual machine with the provided virtual disk image. If you do not want to use the virtual machine, feel free to install **Oracle Database** on your system and do PW2 with that. This is necessary so you have the required permissions to do all of the tasks.

Report Structure

Your report must be a single *.pdf file containing the following sections:

1. **Title Page**: Course title, work title, and the student ID, name, and surname of all group members.
2. **ER Diagram**: A clear ER diagram of your data warehouse (star or snowflake schema) with explanations justifying your design choices (e.g., why you chose a particular schema, what your dimensions, facts, and measures are).
3. **Implementation Reference**: The Oracle server profile where the system is implemented.
4. **Demonstration Video Link**: A link to your group's video demonstration.
5. **Queries and Explanations**: All analytical queries with both semantic (what business question it answers) and technical (how the SQL works) explanations.
6. **Group Work Report**: A brief reflection on your group's process. How did you divide the work? What went well and what didn't? What challenges did you face?
7. **Conclusions**: A summary of your work, discussing the most interesting or challenging design and implementation details. This is a critical part of the report.

The Tasks in the Practical Work

Your goal is to design, build, populate, and query a data warehouse based on your given domain.

1. Design the Data Warehouse (20 points)

Create an ER diagram using a CASE tool or diagramming software.

Minimal Requirements:

- Design a star schema with one central fact table.
- Clearly define at least three dimension tables and one fact table.
- Identify the primary keys, foreign keys, and at least three measures (facts) in the fact table.

Recommended Requirements (for higher evaluation):

- Justify the choice between a star and a snowflake schema and implement one.

- Design and implement dimension hierarchies (e.g., Time: Day → Month → Year; Location: City → Country → Continent).
- Consider and implement a strategy for at least one Slowly Changing Dimension, for instance, Type 2 (creating new rows for changes).

2. Implement the Warehouse and Populate Data (ETL) (10 points)

Write the DDL (CREATE TABLE) statements to implement your design and populate your warehouse with realistic data.

Minimal Requirements:

- Use INSERT ... SELECT statements to populate your dimension and fact tables.
- Each dimension table must have at least **6-10 rows**. The fact table must contain at least **50 rows** to allow for meaningful analysis.

Recommended Requirements:

- Create a PL/SQL procedure to automate the data loading process.

3. Create Advanced Analytical Queries (25 points)

Write queries to retrieve meaningful information from your warehouse. This task combines advanced SQL analytics with custom PL/SQL functions to solve complex business problems.

Minimal Requirements:

- Write four meaningful queries using ROLLUP, CUBE, or GROUPING SETS to perform multi-level aggregation.
- Create two different PL/SQL functions that perform a business-relevant calculation (e.g., calculating a discount, categorizing a customer based on spending, converting units).
- Demonstrate the use of at least one of these functions within one of your main analytical queries (e.g., in the SELECT list or GROUP BY clause).
- All queries must have clear semantic and technical explanations.

Recommended Requirements (for higher evaluation):

- The functions are non-trivial, using conditional logic (IF/ELSE or CASE) or processing multiple input parameters to produce a result.
- Write two meaningful queries that answer non-trivial business questions by combining techniques (e.g., a window function inside a CUBE query, or a PL/SQL function used in a GROUP BY).
- Demonstrate the use of the MODEL clause for complex spreadsheet-like calculations.

Task 4: Design an Aggregate Table (15 points)

High-level dashboards (e.g., a "Yearly Sales" chart) are often too slow if they query a fact table with millions of rows. The solution is an aggregate table (also called a summary table). This task requires you to design one.

Minimal Requirements:

- **Identify**: In your report, identify a key business metric that is frequently queried and would be slow to calculate (e.g., "total monthly sales by region").
- **Design**: Design a new summary table (e.g., Agg_Monthly_Sales_by_Region) that pre-calculates and stores this data.
- **Write the DDL**: Provide the CREATE TABLE statement for this new aggregate table.
- **Write a Sample Query**: Write one analytical query that would run on your new, small aggregate table to get the answer.
- **Justify**: Explain in your report why this aggregate table is useful. Compare the work the database would have to do:
    - Querying the main fact table (scanning 50+ rows, or millions in a real system) vs.
    - Querying your new aggregate table (scanning just a few rows).

Recommended (for higher evaluation):

- **Implement a Population Procedure**: Create a simple PL/SQL procedure that TRUNCATEs (empties) and then INSERT...SELECT...GROUP BYs to populate your new aggregate table with fresh data from the main fact table.
- **Demonstrate Performance Gain**: Use the EXPLAIN PLAN FOR ... command on both your "slow" query (against the main fact table) and your "fast" query (against the aggregate table). Put screenshots of the two plans in your report and compare the estimated “cost”.